横向拆解Claude Code、Codex等六大Agent上下文压缩策略后，我们做了第 7 个

原创 腾讯程序员 腾讯技术工程 2026年6月8日 02:36 广东 441人



作者：mervynyang

> 横向拆解六大 Agent 的上下文压缩策略，提炼通用配方，并面向云端多用户的 Agent 场景落地一套四级水位线方案。

1. 同一个问题，六种做法

Agent 要做上下文压缩，如今几乎成为所有 agent 必做的一环。有意思的是怎么做——我花了不少时间把主流方案扒了一遍，发现大家的分歧大得离谱。

产品	核心策略	一句话概括
Claude Code	五段流水线，按成本递增排列	便宜的本地操作先上，LLM 摘要兜底
Codex CLI	保留近期用户消息原文，其余全部替换为 handoff 摘要	用户说的话最准确，模型说的可以重写
OpenCode	时间戳标记隐藏 + 结构化摘要 + 回放最后一条用户消息	不真删，理论上可恢复
Cline	`/smol`（`/compact`）生成摘要后在同一任务内接续	自动 + 手动双模式
Cursor	自动摘要 + 提示开新对话 + 历史可搜索	压缩后仍能回溯原始历史
Amp	不做递归压缩，用 `/handoff` 开新线程携带要点	长对话本身就是问题，换线程比压缩好
MemGPT / Letta	上下文 = RAM，历史 = 磁盘，Agent 自主换入换出	操作系统级的内存调度

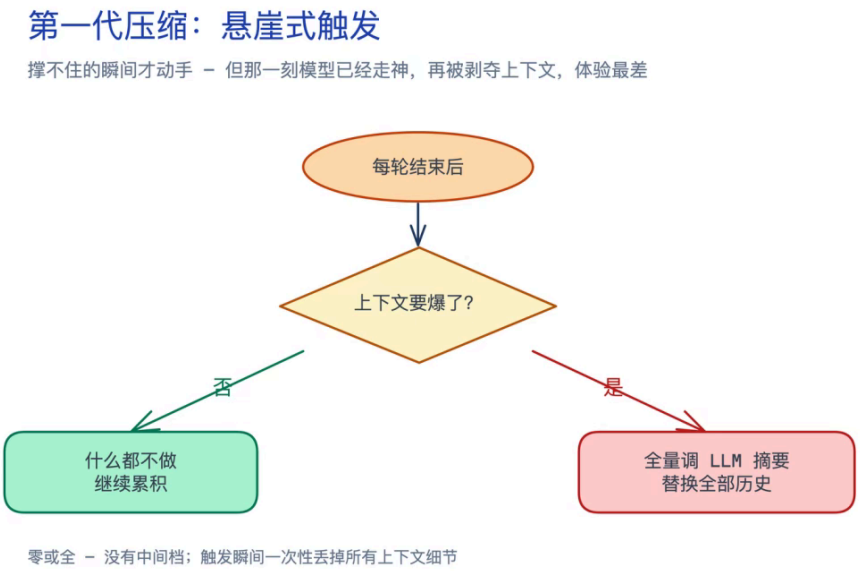
六家产品，六种哲学。说明这件事没有显而易见的最优解，每一种选择背后都是取舍。

后面的内容分三块展开。先横向看看各家的具体做法和设计考量；然后从中提炼几条已经接近共识的原则；最后介绍我们在 MUR AI 上最终落地的方案——四级水位线 + 增量摘要，以及云端多用户场景下额外需要的几层设计。

MUR AI 是一个面向用研场景的云端多用户 Agent。跑在云端、服务多人这两个特点，让我们在压缩方案上比本地 CLI 工具多了好几层要考虑的事，第 6 节会细聊。

2. 反面教材：第一代"撑不住才动手"

要理解新方案为什么长成这样，先得看清它们想摆脱什么。第一代压缩的逻辑一句话能讲完：



直观，好实现。但用过的人都知道体验很差——下面五条是痛点的根源。后面你会发现，新方案各自的设计选择，本质上就是在重点解决其中某一两条。

悬崖式触发。 不溢出的时候一动不动——对话越来越胖、模型注意力越来越散，系统毫无反应。一旦溢出就全量出手，把前面几十轮一口气捏成一段摘要。这种"零或全"意味着触发的那一刻质量已经塌了——模型刚因为信息过载走神，紧接着又被剥夺大半上下文。

全量摘要丢细节。 几十轮消息压成几百字，无论 prompt 写得多好都会丢变量名、函数签名、错误堆栈、用户的具体措辞——偏偏这些是 Agent 继续干活最需要的东西。

Token 估算粗糙。 不少实现用 `text.length / 3` 估算 token。中英混合场景下误差 30-50%，以为安全实际已经溢出，或者以为该压了实际还早。

不区分信息价值。 5000 行 grep 输出和 5000 token 的关键诊断被同等对待——前者裁了不影响任务，后者裁了 Agent 立刻不会干活。

用户内容被一刀切。 用户贴的代码代表输入意图，和工具输出性质不同。但如果一视同仁地压，就会出现"用户贴的代码被压没了，模型忘了要改什么"。

第一代做法的根本问题：它把压缩当突发事件处理，而不是一种持续维护的能力。

3. 第二代：各家的做法

过去一年，几个主流 Agent 不约而同走向"分层 + 渐进"，但味道各不相同。我逐个拆解一下。

Claude Code（Anthropic）：五段流水线 + 结构化摘要

Claude Code 把上下文管理做成了一条严格按成本递增排列的流水线：

1. Budget Reduction——调整工具输出的截断预算
2. Snip——截短老的工具输出，留"做过什么"的摘要行
3. Microcompact——对工具输出内容做局部内联压缩
4. Context Collapse——对更久远的历史做粒度更细的折叠
5. Auto-Compact——兜底，调 LLM 生成结构化摘要

前四步都是纯本地操作，零 API 调用。只有第五步才会请求 LLM。摘要本身是结构化的，固定包含九个章节：用户意图、主要请求、技术概念、文件与代码段、错误与修复、问题解决过程、用户消息、待办任务、下一步。

有个值得注意的细节：它在压缩时刻意保持消息序列前缀稳定，让 Prompt Cache 命中率不会因为压缩而掉下来。

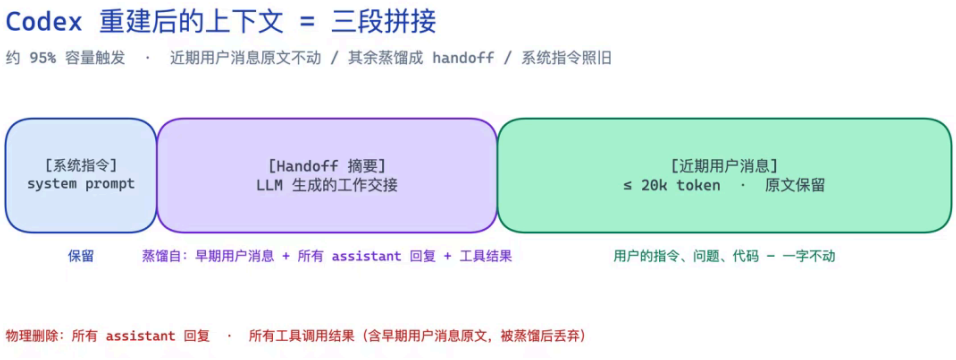
Claude Code 内部其实还有两条更激进的路径，思路都是"把脏活交给服务端，客户端一个字节都别改"：

- cached_microcompact：把"删掉旧 tool 结果"包装成 API 层的 cache_edits 指令。客户端发出去的 prompt 原封不动，服务端在已缓存的前缀上直接抠掉指定内容——字节没变，缓存不失效。
- apiMicrocompact：更彻底，直接调 Anthropic 的 context_management API（beta context-management-2025-06-27，Vertex / Bedrock 也支持），让服务端按 input_tokens 阈值自动裁剪旧的工具调用。

换句话说，前面那条本地五段流水线在很多场景下已经退居二线了。能让服务端做的就让服务端做，本地操作永远是兜底。

Codex CLI（OpenAI）：近期用户消息优先保护

Codex 的策略相对简单——在约 95% 容量时触发，生成一份 handoff 摘要替换掉旧历史。重建后的上下文只剩三部分：



所有 assistant 回复和工具结果被物理删除，由摘要替代。近期约 20k token 内的用户消息原样保留，更早的用户消息则被蒸馏进摘要里——关键请求和约束会被保留下来。

设计哲学是把压缩当作"同事间的工作交接"：进展、约束、剩余任务，足够下一个模型接手继续干。

OpenCode：可逆隐藏 + 回放最后一条指令

OpenCode 的做法我觉得很有意思，它分两步走：

第一步 Prune（轻量，无 LLM 调用）：每次成功响应后自动触发。往回遍历消息，跳过最近 2 轮用户对话，保护最近 40k token 的工具输出不动，把更老的工具输出用时间戳标记为"已压缩"。关键是——数据还在数据库里，只是模型看到的变成了占位符 `[Old tool result content cleared]`。

第二步 Summary（重量，调 LLM）：只在 token 用量超过模型输入上限时触发。生成一份五段式结构化摘要（目标 / 指令 / 发现 / 已完成 / 相关文件），然后自动回放用户最后一条消息——模型不会从摘要继续，而是从你最近的指令继续。

值得一提的是，它只有一份摘要同时服务于模型和界面展示，并没有做"给模型看的详细版"和"给用户看的精简版"的分离。

Cline：自动 + 手动双模式

Cline 从 v3.25 开始有两种压缩模式：

- `/smol`（别名 `/compact`）：手动触发，生成摘要后在同一任务内接续。决策、代码变更、状态都保留在摘要里，不用切新会话。
- Auto-Compact：接近上下文上限时自动触发，行为和 `/smol` 一致。

如果开启了 Focus Chain（v3.25 默认开启），待办列表会穿越压缩存活下来，作为进度锚点。

Cursor：压缩 + 可回溯

Cursor 在上下文超出模型窗口时自动压缩旧消息，同时会提示用户"开一个带摘要的新对话"。

2026 年他们加了个有意思的能力——Dynamic Context Discovery：把聊天历史变成可搜索的文件，即使压缩后 Agent 也能回头检索原始细节。据说在 A/B 测试里减少了 46.9% 的总 token 消耗。

不过社区反馈里有个已知问题：压缩后模型有时会"忘掉"刚才的编辑，Cursor 团队确认这是高优 bug 在修。

Amp（Sourcegraph）：不压缩，换线程

Amp 的立场很鲜明：递归摘要会导致性能逐步衰减（他们引用了 OpenAI 的一份内部研究），所以干脆不做。替代方案是 `/handoff`——把当前线程的要点打包进一个新线程，用户可以在

交接前审查和编辑带过去的内容。

线程在 Amp 里是一等公民：可以用 `@@` 引用其他线程、用 `threads: map` 可视化线程关系。理念是"一系列有焦点的短步骤，比一个逐渐退化的长对话好"。

不过 2026 年的 Neo CLI 更新里，Amp 也加了 90% 窗口用量时的自动上下文管理——算是对纯手动路线的一个妥协。

MemGPT / Letta：上下文当 RAM

学术派的代表。直接按操作系统的内存层次来建模：

层级	类比	容量	访问方式
Main Context	RAM	模型上下文窗口大小	始终在 prompt 里
Recall Memory	交换分区	完整对话历史	`conversation_search`
Archival Memory	磁盘	无限（向量存储）	`archival_memory_search`

关键区别是：换入换出由 Agent 自己决定（通过函数调用），不是被动截断。Letta 是 MemGPT 的生产化框架，最新版本 v0.16.7（2026 年 3 月），GitHub 22.5k star，还在积极维护。

代价是架构复杂度高、需要外部向量存储、有检索延迟。适合需要跨会话长期记忆的场景，但对单会话内的压缩来说有点重。

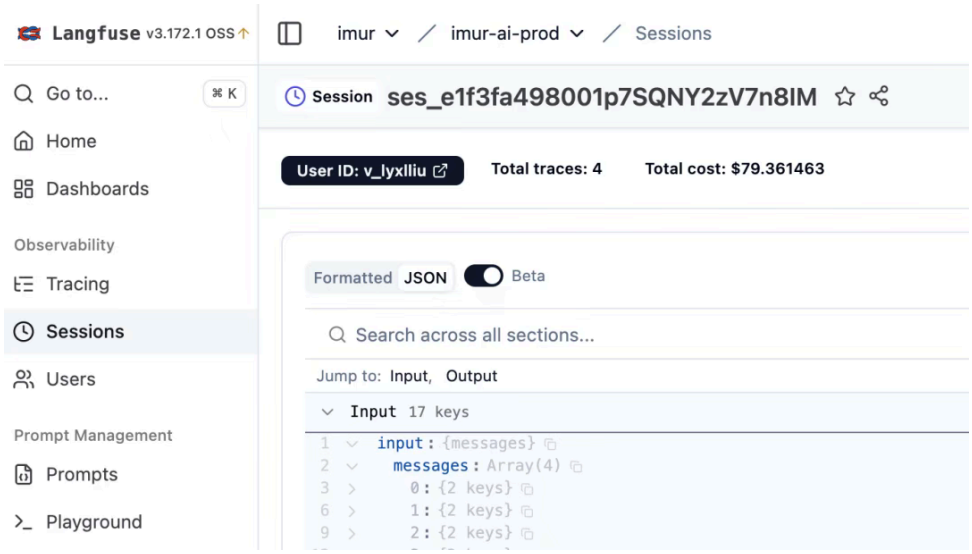
3.x 第二代的实施陷阱：滑窗式 stub 替换 = 每步缓存失效

> 2026-05-13 补

第二代"分层渐进"如果实施得不对，会掉进另一个隐蔽的坑。

想象这样一种实现：你设了一个规则——"保留最近 N 条 tool 结果，更老的替换成 stub"。听起来很合理对吧？问题在于如果这个判断在 step-loop 里每一步都重算，那么每完成一个 step（新增 2 条消息），就有 1 条原本被保留的旧 tool 结果滑出窗口、被替换成 stub。它的字节变了，从这个位置往后的整段 prompt 前缀对 Prompt Cache 就失效了，需要重新写入。

把压缩塞进每个 step 里跑、还用滑窗——结果就是每个 step 缓存前缀都会失效一次。



我们之前有一个真实的 Task：一个 4 轮、177 step、59 分钟的会话，烧了 $77.3，其中 **83%（$64.8）全是 cache_write**。要知道 cache_write 的单价是 cache_read 的 12.5 倍。通过 Langfuse 中的 step 数据，一目了然：

step	nMsgs	#stub	lastStubIdx	cacheRead	cacheWrite	
5	188	75	165	175958	295	← 命中
6	190	75	165	176253	232	← 命中
7	192	76	167	102795	74555	← 炸了
8	194	77	169	102795	77238	← 又炸
9	196	78	171	102795	77634	← 还炸
...						
58	294	101	273	102795	117088	← 一直炸

看规律：`#stub` 每 step 加 1，`lastStubIdx` 每 step 加 2，`cacheRead` 永远卡在一个小数不动，`cacheWrite` 一路涨到约 12 万/step。从 step 7 起前缀缓存就死了，剩下 50 多个 step 每一步都在为"窗口又挪了一格"买单。

stub 决策必须单调推进——只能大跳，不能滑窗。一个 part 一旦被标成 stub，后续所有 turn、所有 step 里都保持 stub 不变，绝不因为"又老了一步"而反复触发。上面提到的 `cache_edits` API（Claude Code 走的那条路）天然满足这个性质——客户端字节不改，决策在服务端生效，缓存前缀永远稳定。

4. 从实践里提炼出的共识

方案放在一起看，细节差异很大，但有几条原则几乎人人认同：

分层渐进，不一刀切。 定义多个水位线，越接近上限手段越激进。系统永远在小幅维护，避免悬崖式塌方。

成本严格递增。 便宜的先做（字符串截断、placeholder 替换），贵的最后做（LLM 摘要）。能用零成本释放的空间，不花钱买。

增量摘要优于全量摘要。 老做法每次重新摘要全部历史，新做法保留一份活的摘要，每次只把新增部分合并进去。好处：单次输入更短更便宜也更准；同一段历史不会被反复重写，避免"摘要的摘要"导致语义漂移；合并时模型可以主动取舍（"这个文件改过了，旧描述更新一下"）。

用真实 token，别估算。 LLM API 每次都返回 `usage.totalTokens`，免费、精确、唯一可信。`text.length / 3` 只在内部排序时凑合用（"先裁哪个工具输出"），触发判断必须用真实值。

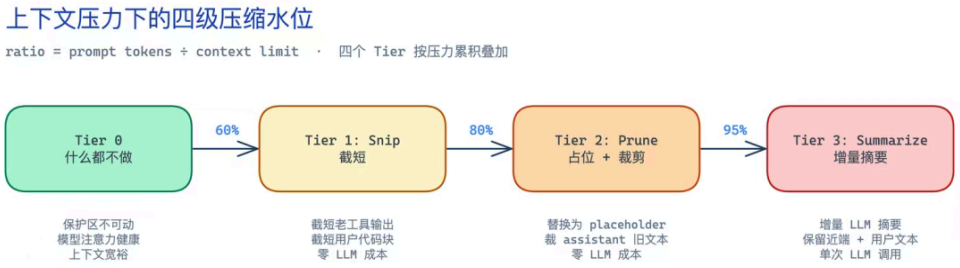
用户消息有特权。 用户的指令、问题、代码——这些是任务来源。Codex 做到一字不动，OpenCode 做到压缩后回放最后一条，其他方案至少保证用户纯文本不裁。

保护近端。 无论怎么压，最近几轮不能动。模型短期连贯性几乎全靠这几轮维持。常见做法是定义保护区——比如最近 8000 token 内的所有消息，任何级别都不参与压缩。

单调边界，绝不滑窗。（2026-05-13 补）stub 决策一旦做出，对应位置的字节就必须从此固定——下一 turn、下一 step 再看到同一位置，永远是同一个 stub，绝不因为"又老了一步"而重新触发替换。实现上有两种常见做法：把决策按 part ID 持久化（Redis 或内存映射），下次直接复用不重算；或者干脆把这件事交给服务端的 `cache_edits` / `context_management` API，客户端字节零变化，缓存天然稳定。

5. 我们的方案：四级水位线

把上面的原则落地，我们选了四级水位线。理解方式很简单——想象一台电脑的内存压力监视器：



四个 Tier 不是互斥而是累积——Tier 3 触发时会先做完 Tier 1 和 Tier 2 再做摘要。这意味着即使最坏情况，需要送给 LLM 的内容量也已经被前两步免费砍掉一大块。

Tier 0：什么都不做（< 60%）

上下文宽裕，模型注意力没散，最好的优化是不优化。

Tier 1：Snip——便宜的整理（60-80%）

到了 60% 开始预防性维护，没有 LLM 调用，纯字符串处理：

- 截短老的工具输出。一次 grep 返回 5000 token？保留前几行 + 工具名 + "还有 X 条结果被省略"，剩下丢掉。

- 截短用户消息里的代码块。200 行代码保留文件名注释 + 前几行 + 总行数标注。

几个细节：保护区内的工具输出和代码块不动；某些工具享有豁免（比如 Skill、Task 这种返回结构化关键信息的）；用户的纯文本指令永远不动，只压缩 markdown 代码块。

这一级成本是零，但能挡住相当一部分增长。

Tier 2：Prune——更狠的释压（80-95%）

预防性维护不够了：

- Tier 1 已经截短的工具输出进一步替换成占位符（`[Content compacted to save space]`）
- 裁掉 assistant 旧文本——保留前两句 + `[truncated]`
- 截断阈值整体下调，能压的全压

依然零 LLM 成本，依然不动保护区，依然不动用户纯文本。

Tier 3：Summarize——兜底（≥ 95%）

只有 Tier 1 + Tier 2 都救不回来时才触发 LLM 摘要。做的是增量摘要：

1. 找出"上次摘要之后 ~ 保护区之前"的消息作为 delta
2. LLM 输入：上次摘要 + delta → 生成合并摘要
3. 替换旧摘要，删除 delta 消息
4. 保护区不动

第一次触发时"上次摘要"为空，相当于普通摘要；之后每次都是追加合并，避免反复重写历史导致语义漂移。

摘要 prompt 用结构化输出，四段：进展 / 文件 / 待办 / 上下文（用户偏好、错误、约束）。

6. 云端 Agent 还要多做三层

前面四级水位线解决的是"上下文里压什么、压多狠"。但 MUR AI 跑在云端、服务多用户——这意味着我们得处理几件 CLI 工具可以无视的事：

- 用户关掉浏览器再回来，压缩状态不能丢
- Pod 重启、流量漂移，跨进程的压缩决策必须一致
- 工具完整日志要支持事后审计和前端回取，不能为了省 context 就把日志丢了
- sandbox 里的工具输出动辄几十 MB，完整性和模型注意力不能二选一

这些靠水位线解决不了。所以在四级水位线之上，我们叠了三层云端特化设计。

6.1 存储分离：完整日志落盘，对话里只留截断版

每次工具调用——bash、read、grep——都可能吐出几万 token。直接塞进对话历史不行，全

丢掉又损失调试和审计能力。

我们的做法：完整输出落盘，对话里只保留截断版 + 一条回取路径。

具体来说：engine 层调用 `persistTruncatedOutput` 把完整内容写到沙箱的 `_internal/truncated-outputs/{callId}.log`；沙箱写失败就降级到 COS 直传。截断版的 metadata 带上 `fullLogPath`，模型看到的是"前几行内容 + [截断] + 完整日志路径"——它知道完整内容在哪，但没花 token 去读。前端展示时按需调 sandbox-file API 现取现读，完全绕开 context 约束。

本质上是把"模型的工作记忆"和"用户的审计需求"解耦了。CLI 工具不用管这个——它们没有前端，工具输出要么留在 context 里，要么就没了。

6.2 工具差异化：不是所有工具都该同等对待

第 5 节提过"某些工具有豁免权"，这里把完整分类讲清楚。我们把工具分四个梯度：

- 完全保护（`PROTECTED_TOOLS = ['Skill', 'Task']`）：Skill 关联知识库绑定，Task 是会话级元任务。输出有状态意义，删了 Agent 就糊涂了。任何 Tier 都不动。
- 微压缩豁免（Task / AskUserQuestion）：被动小幅压缩也跳过。AskUserQuestion 的输出是用户的回答，删了等于把用户回话抹掉。
- 白名单内可压（bash / read / grep / websearch …）：无状态读取类工具，压缩的主力。
- 差异化存储预算：单次输出落盘上限按工具区分——Read 30KB、Bash 50KB、WebSearch 15KB。Bash 经常吐大段构建日志，Read 相对可控，WebSearch 通常只要几条结果就够。

这里的考虑很直接：一段 grep 输出和一次 Skill 调用的信息密度不在一个量级，用统一阈值处理就是粗暴。

6.3 跨轮缓存：让压缩决策在重启后还算数

这是云端特有的问题。

场景：用户聊到第 20 轮，系统在第 8 轮、第 14 轮分别做过 snip，截短了某些 grep 输出。然后实例重启了，或者下一个请求被路由到另一个 Pod。第 21 轮触发新一轮压缩——如果什么都不记得，新进程会从头重新判断每个 part 该怎么压，结果可能跟之前不一样。

这会带来两个问题：

- Prompt Cache 全废——消息序列前缀变了，缓存命中率掉到零，每一轮都按新 prompt 计费
- 模型困惑——同一段历史在不同轮次里"长得不一样"，模型可能重新触发已经处理过的工具调用，甚至开始鬼打墙

我们的解法叫 ReplacementCache——把每一轮的截断决策按 part ID 存进 Redis（key 形如 `msgOptCache:{sessionId}`，TTL 30 分钟）。下一轮无论是同进程还是另一个 Pod，先查缓存。已经决定过怎么压的 part 直接复用之前的结果，没决定过的才走新策略。

效果：同一个 part 在整个会话里始终长一样，消息前缀稳定，Prompt Cache 友好；跨实例、跨重启无感；从缓存读出的数据先过 `isValidReplacementEntry()` 校验，损坏的直接丢弃重算。

6.4 多用户隔离

最后一层最朴素但绝对不能省。所有压缩状态按 `(userId, sessionId)` 二元组隔离：

- 数据库写入强制带双条件 `WHERE userId = ? AND sessionId = ?`，杜绝越权读写
- COS 路径形如 `user_sessions/{userId}/sessions/{sessionId}/_internal/...`，天然按用户分区
- SSE 压缩事件按 sessionId 严格过滤订阅，一个用户收不到另一个用户的压缩通知

多租户场景下，"压错给谁"比"压错什么"严重得多。这一层没有为了性能而省的空间。

这四层——存储分离、工具差异化、跨轮缓存、多用户隔离——是 CLI 工具都不必处理的。它们是单机工具，重启意味着开新会话，压缩状态丢了就丢了。但云端 Agent 不一样：用户合上电脑去吃饭、明天回来，期待的是接着上次继续，而不是"对不起请重新介绍一下需求"。

7. 几个关键决策背后的原因

设计过程中有几个看似细节但影响很大的选择。

为什么阈值是 60 / 80 / 95?

60% 是预防性维护的甜点——再低会频繁触发没意义，再高模型注意力已经开始下降。80% 是危险线，离溢出还有缓冲但已经不能等。95% 是最后防线，再不调 LLM 就要爆了。

这三个值都是可配置的常量，接入远程配置后能热更新。出问题时把任意一个调到 1.0 就能禁用对应 Tier。

为什么先 Snip 再 Prune?

Snip 保留更多元信息。一个被 snip 的工具结果还能让模型看到"我之前 grep 过这个文件"；被 prune 替换成占位符后连这个都看不到了。Snip 是轻量记号，Prune 是彻底擦除——能用前者解决的就别上后者。

为什么要增量摘要?

打个比方：

- 全量摘要 = 每周把过去三个月的工作重新写一份周报
- 增量摘要 = 维护一份持续更新的项目状态，每周只追加和修订变化的部分

哪种更准更便宜？不言自明。增量摘要还有个隐藏好处：同一个文件被多次提及时，最新状态覆盖旧状态；全量摘要里这个文件可能被描述好几次而且互相矛盾。

为什么用真实 token?

我们一开始也用 `text.length / 3` 估算。直到有一次发现估算值显示 70%，实际 LLM 返回的 usage 已经 92%——再来一轮直接溢出。原因是中文字符在 BPE 里通常占 1.5-2 token，英文代码大量短 token，混合内容的实际 token 数比字符估算高 30-50%。

LLM API 每次都返回精确的 `usage.totalTokens`——免费、精确、唯一可信，没理由不用。但内部排序（"先裁哪个工具输出"）继续用估算就够了，需要的是相对大小不是绝对值，30% 误差不影响排序。

为什么保留 compactionProtected 标记?

预留扩展。某些 Part 可能很重要但看起来很普通——用户上传的设计文档、关键错误堆栈、标记为"记住这个"的指令。给它们打标记，任何 Tier 都跳过。能力先建好，需要时直接用。

8. 红线：什么东西任何 Tier 都不能动

把红线列清楚很重要——压缩系统最大的事故不是压不够，而是压错东西。

内容	原因
保护区内的所有消息	模型短期连贯性的命脉
用户消息的纯文本部分	用户意图就是任务来源
PROTECTED_TOOLS 的输出（Skill / Task）	高度结构化的关键信息或会话级状态
MICRO_COMPACTION_EXEMPT 的工具（Task / AskUserQuestion）	保住对话流的关键节点
带 compactionProtected 标记的 Part	业务侧明确指定的"必须保留"

这些规则在每一级压缩里严格执行。不存在"为了救场破例"的情况。如果保护区都救不下来，那就让上下文真的爆出来——比错误压缩造成模型行为漂移要好。

9. 可观测性：让压缩被看见

压缩发生在背后，没有可观测性的话调起来全靠玄学。我们在 SSE 事件里加了这些信息：

- 当前触发的 tier（0 / 1 / 2 / 3）
- 当时的 token 使用率（来自 LLM 真实 usage）
- Snip 截了几个 part
- Prune 替换了几个 part
- Summarize 是否真的调用了
- 预估节省了多少 token
- 命中 ReplacementCache 的 part 数（衡量跨轮一致性收益）

前端可以据此渲染一个压缩面板：让用户看到"这一轮触发了 Tier 1，节省了 3000 token，其中 6 个 part 直接复用了上一轮决策"。这些数据接入 trace 系统后，可以基于真实生产数据调水位线，不再拍脑袋。

10. 我们没做的事

为了避免过度工程，有几条路我们暂时没走，但保留了可能性：

- **主动 cache-aware 调度**（Claude Code 的做法）：压缩时主动调整顺序来最大化 Prompt Cache 命中。我们目前用 ReplacementCache 实现了被动一致性，还没做按 cache 边界主动选择压谁。等 cache 命中率成为瓶颈再上。
- **可逆隐藏**（OpenCode 的做法）：用时间戳标记而非真删。等用户需要"回滚到某次压缩之前"时再上——目前 `_internal` 落盘已经满足审计需求。
- **回放最后一条用户消息**（OpenCode 的做法）：摘要后重新追加用户最近指令。Tier 3 触发频率本来就低，优先级不高。
- **用户消息一字不动**（Codex 的做法）：我们已经不动用户纯文本，但还会截断用户贴的代码块。是否进一步走极端，看后续生产数据。
- **分层长期记忆**（MemGPT / Letta / Mem0）：跨会话记忆是另一个产品方向，不是这一轮要解决的问题。

11. 说点感性的

聊了这么多技术细节，最后跳出来说一件事。

Context 压缩的目标从来不是省 token。省钱是顺带的。它要解决的问题是保护模型的注意力。

200K 的上下文窗口听起来很大，但研究反复表明：上下文塞到 70% 以上，模型的中段失忆和指令漂移就会明显恶化。它不是真的"忘了"，是注意力被稀释、信号被噪声淹没。这就是 Context Rot。

所以一个合格的压缩系统应该是一个信号工程师——把无关紧要的工具输出降为占位符，让模型不用扫过它们；把老的 assistant 文本裁短，让最近的对话不被淹没；把历史合并成结构化摘要，让模型用事实思考而不是用文本回忆。

我们花力气做分级、做增量、做保护区，本质上是在回答一个问题："这一轮对话里，模型应该把注意力放在什么上面？"

2026 年做 Agent 工程，这个问题绕不开。

附录：参考资料

- Anthropic — Effective context engineering for AI agents
- Anthropic — Compaction (Claude API)

- Justin3go — Shedding Heavy Memories: Context Compaction in Codex, Claude Code, and OpenCode
- badlogic — Context Compaction Research: Claude Code, Codex CLI, OpenCode, Amp
- Letta — Agent Memory: How to Build Agents that Learn and Remember
- Mem0 — LLM Chat History Summarization Guide
- Microsoft — Compaction (Agent Framework)
- MemGPT — Engineering Semantic Memory through Adaptive Retention
- Cline / Cursor / Amp — 各家官方文档



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 2.4万

留言

写留言

还没有留言，来写首评